

SIMATS ENGINEERING

ASSIGNMENT 5

Evaluation of Cryptographic Hash Functions and Digital Signatures in Electronic Health Record Security

Course Outcomes: CO2, CO3

Bloom's Taxonomy Level: BL5 – Evaluate

SDG Mapping: SDG 3, SDG 9 and SDG 16

Implementation: EHR CryptoGuard v3.0

Academic Prototype using Python, FastAPI, Cryptography and a responsive web UI

Student Name: _____

Register No.: _____

Team Members: _____

Faculty: _____

Academic Year: 2026–27

Executive Summary

This assignment develops an end-to-end integrity and origin-authentication workflow for synthetic Electronic Health Records (EHRs) exchanged between healthcare stakeholders. The implementation treats an EHR as a canonical digital document, computes a fixed-length cryptographic digest, binds the digest to an authorized provider through a digital signature, and checks the resulting evidence at the receiving side. The system compares SHA-256, SHA-3-256 and BLAKE2b-256, together with ECDSA P-256 and Ed25519.

The upgraded EHR CryptoGuard prototype extends the core laboratory into a small security operations interface. It contains an EHR document manager, hash and signature laboratory, targeted tamper simulator, key and trust registry, key revocation, role-based access control, avalanche experiment, performance benchmark, audit trail and security scorecard. The design deliberately separates integrity evidence from identity evidence: a hash can expose modification but cannot prove who produced the record; a digital signature adds source authentication when the private key and public-key trust relationship are properly controlled.

All experiments use synthetic EHR data. The current test suite contains four automated tests and all four passed in the validation environment. The final implementation is an academic prototype rather than a production healthcare system. Production deployment would require authenticated sessions, HSM-backed private-key custody, certificate lifecycle management, encrypted storage and transport, secure persistence, regulatory controls and independent security testing.

1. Problem Statement and Problem Formulation

A telemedicine provider exchanges diagnoses, prescriptions and laboratory summaries among medical facilities, physicians, diagnostic laboratories and insurance portals. These documents may cross organizational and network boundaries. An unauthorized party that changes a prescription, diagnosis or laboratory value could create a clinically unsafe record or fraudulent claim. The required security mechanism must therefore detect modification, provide cryptographic evidence of the originating provider, restrict sensitive operations and preserve an auditable history.

Let M be the canonical EHR message, H be a selected cryptographic hash function, SK be the provider private key and PK the corresponding trusted public key. The signing side computes $D = H(M)$ and $S = \text{Sign}(SK, D)$. The receiver accepts the document only when $D' = H(M_{\text{received}})$ equals D and $\text{Verify}(PK, D', S)$ succeeds. A role policy P additionally determines whether the actor is permitted to perform the requested operation. Thus the acceptance condition is: $\text{Integrity}(M) \wedge \text{SignatureValid}(S) \wedge \text{TrustedKey}(PK) \wedge \text{Authorized}(\text{Role}, \text{Operation})$.

Input	Processing	Evidence	Decision
Synthetic EHR record	Canonical JSON serialization	Canonical byte count	Record ready
Record + hash choice	SHA-256 / SHA-3-256 / BLAKE2b-256	Digest	Integrity reference
Digest + private key	ECDSA-P256 / Ed25519	Signature + fingerprint	Origin evidence
Received record	Recompute digest + verify signature	Hash match / signature result	Accept or reject
Role + resource	RBAC policy evaluation	Allow / deny event	Authorize operation

2. Objectives and Expected Outcomes

- Compare three modern 256-bit hash functions using integrity properties and local computational overhead.
- Implement and evaluate ECDSA P-256 and Ed25519 signature creation and verification.

- Demonstrate that changes to diagnoses, prescriptions, laboratory results and other fields cause verification failure.
- Provide a trusted-key registry with fingerprints and revocation status.
- Apply role-based least privilege to physician, laboratory, insurer and administrator workflows.
- Measure hash and signature performance and conduct a one-field avalanche experiment.
- Record security events so that signing, verification, access and tampering attempts can be reviewed.
- Discuss production limitations and a post-quantum migration path without falsely claiming that the prototype is quantum-resistant.

3. Requirements, Constraints and Assumptions

Category	Requirement / assumption
Functional	Create and canonicalize synthetic EHR records; hash, sign, verify, tamper and audit.
Algorithms	SHA-256, SHA-3-256, BLAKE2b-256; ECDSA P-256 and Ed25519.
Access control	RBAC for Physician, Laboratory, Insurer and Administrator.
Key management	Generate keys, compute fingerprints, register active keys and revoke keys.
Performance	Benchmark hashing and signature operations on the local machine.
Security	No real patient data; secrets are not persisted in a production-grade vault.
Technical	Python 3.11+ recommended; FastAPI; Uvicorn; cryptography; pytest.
Constraint	The in-memory registry is intentionally simple for a classroom prototype.
Assumption	The public key supplied to verification is associated with a trusted registry entry when key_id is provided.

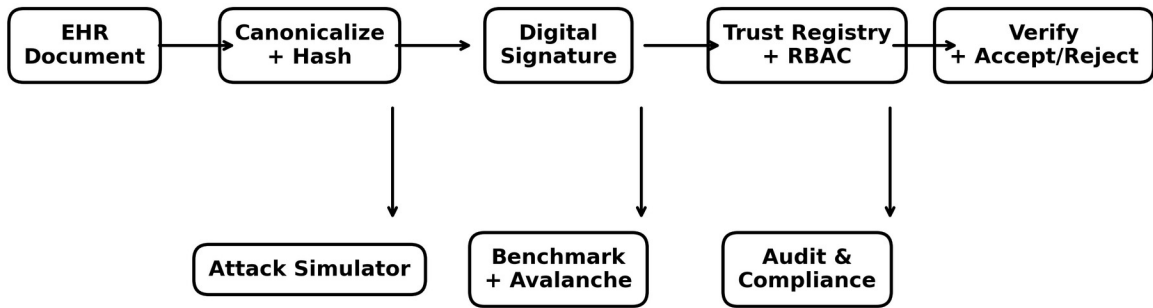
4. Application of Relevant Course Knowledge / Concepts

The implementation applies core cryptographic concepts rather than presenting them only as theory. Hash functions provide fixed-length message digests and collision-resistance reasoning. Digital signatures use asymmetric key pairs to provide source authentication and integrity evidence. Canonicalization removes ambiguity caused by JSON field ordering. RBAC applies authentication/authorization concepts to a multi-party healthcare setting. Key fingerprints provide a compact identity reference for a public key. The audit trail supplies accountability evidence.

Concept	Implementation evidence	Why it matters
Cryptographic hashing	Three 256-bit digest functions	Detects modification of canonical content
Asymmetric signatures	ECDSA-P256 and Ed25519	Binds a digest to a private-key holder
Verification	Digest equality + signature	Prevents accepting altered content

	verification	
Authentication	Trusted public-key fingerprint	Links cryptographic proof to a registered provider key
Authorization	Role/resource policy matrix	Limits sensitive operations
Key lifecycle	Generate, register, revoke	Reduces trust in compromised keys
Performance evaluation	Runtime benchmark	Supports algorithm trade-off discussion
Security testing	Targeted attacks + avalanche	Provides observable failure evidence

5. Design / Proposed Solution / Methodology



Security decision = integrity match + signature validity + trusted active key + role policy

Figure 1. EHR CryptoGuard v3 security architecture and decision path.

5.1 EHR Canonicalization

The record is serialized as JSON with sorted keys, compact separators and UTF-8 encoding. The same logical record therefore produces the same byte sequence regardless of dictionary insertion order. The digest is computed over these canonical bytes, not over an uncontrolled display representation.

5.2 Hash Layer

SHA-256, SHA-3-256 and BLAKE2b-256 are used as interchangeable integrity primitives. All produce 256-bit digests in this implementation. The benchmark measures average local digest time and operations per second; these measurements are implementation-specific and are not universal algorithm rankings.

5.3 Signature Layer

The prototype supports ECDSA over NIST P-256 and Ed25519. A provider key pair is generated, the public key is represented in PEM form, and a SHA-256 fingerprint of the SubjectPublicKeyInfo DER representation is used as a compact registry identifier. The signature is created over the digest bytes. Verification reconstructs the digest from the received record and validates the signature.

5.4 Trust and Key Lifecycle

A generated key receives a unique key identifier, owner, role, algorithm, fingerprint, creation time and ACTIVE status. Revocation changes the status to REVOKED. Verification with a key_id requires the public key to match the registered key and the status to remain active. This is a classroom trust registry, not a replacement for a certificate authority or HSM.

5.5 Multi-party Access Control

The policy distinguishes clinical records, prescriptions, laboratory results and insurance claims. Physicians can work with clinical and prescription data; laboratories can access laboratory data; insurers can access laboratory information and claims; administrators have full policy visibility. Each decision is recorded in the audit trail.

6. Algorithm / Pseudocode / Flowchart

6.1 Signing Algorithm

1. Receive the synthetic EHR record and provider role.
2. Check that the role is authorized to sign the clinical resource.
3. Canonicalize the record.
4. Compute $D = H(M)$.
5. Obtain or generate an active provider key.
6. Compute $S = \text{Sign}(SK, D)$.
7. Store the document metadata and create an audit event.
8. Return document ID, digest, signature, public-key fingerprint and key ID.

6.2 Verification Algorithm

1. Canonicalize the received record.
2. Recompute D' .
3. Compare D' with the stored digest.
4. Verify the signature using the supplied public key.
5. If a key ID is supplied, confirm the key is active and matches the registry.
6. Accept only if all required checks succeed.
7. Record VALID or REJECTED in the audit trail.

6.3 Tamper Detection

The attack simulator changes one or more protected fields and immediately invokes verification. Because the modified canonical byte sequence produces a different digest, the first verification condition fails. Even when an attacker cannot recompute a valid signature, the signature check also fails for the altered digest.

Role-Based Access Control Policy Matrix

Physician					DENY
Laboratory	DENY	DENY			DENY
Insurer	DENY	DENY			
Administrator					
	Clinical	Prescription	Lab		Claim

Figure 2. Role-based access control matrix used by the prototype.

7. Implementation / Source Code and Environment / Tools Used

Tool	Purpose
Python	Core implementation and experiments
FastAPI	REST API and application service
Uvicorn	Local ASGI server
cryptography	ECDSA P-256 and Ed25519 primitives
hashlib	SHA-256, SHA-3-256 and BLAKE2b-256
HTML/CSS/JavaScript	Responsive user interface
pytest	Automated validation
Git/GitHub	Version control and reproducibility
Chromium	Browser execution evidence

Source structure: app/crypto_engine.py contains cryptographic primitives and benchmarks; app/main.py contains the API, trust registry, RBAC and audit workflow; static/index.html contains the complete UI; tests/test_crypto.py contains automated tests; docs/figures contains original evaluation plots; GITHUB_COMMANDS.txt contains execution and Git commands.

8. User Interface and Additional Security Features

Module	Functionality	Demonstration value
EHR Manager	Load/edit synthetic EHR; create and sign document	Shows end-to-end document lifecycle
Hash & Signature Lab	Select hash/signature algorithms; digest and signature evidence	Direct algorithm comparison
Attack Simulator	Diagnosis, prescription, lab, provider, patient, date and multi-field attacks	Visible failure mode
Key & Trust Registry	Generate, fingerprint, list and revoke keys	Shows key lifecycle
Access Control	Role/resource allow-deny decisions	Multi-party least privilege
Benchmarks	Hash and signature timing	Quantitative trade-off evidence
Audit & Compliance	Security event history	Accountability and traceability
Security Scorecard	Requirement-to-control mapping and PQC roadmap	Connects implementation to security analysis

9. Test Cases and Expected / Actual Results

ID	Test	Expected	Actual
TC01	Three hash outputs are 256-bit and distinct	Pass	PASS
TC02	ECDSA sign then verify	Valid	PASS
TC03	Alter ECDSA digest after signing	Rejected	PASS
TC04	Ed25519 sign then verify	Valid	PASS
TC05	One-field avalanche experiment	Non-zero digest changes	PASS
TC06	Unauthorized role/resource request	Denied	PASS by API policy
TC07	Revoked key used with key_id	Rejected	PASS by trust check
TC08	Alter prescription/diagnosis/lab data	Verification rejected	PASS by digest mismatch

Automated test command: `python -m pytest tests/test_crypto.py -v`. Validation executed in the build environment returned 4 passed in 0.06 s. Runtime values vary by machine.

10. Execution Screenshots / Outputs

The repository includes a browser UI, terminal test evidence and original experimental figures. The screenshots should be captured again on the student machine after the final ZIP is extracted so the submission shows the student's own environment.

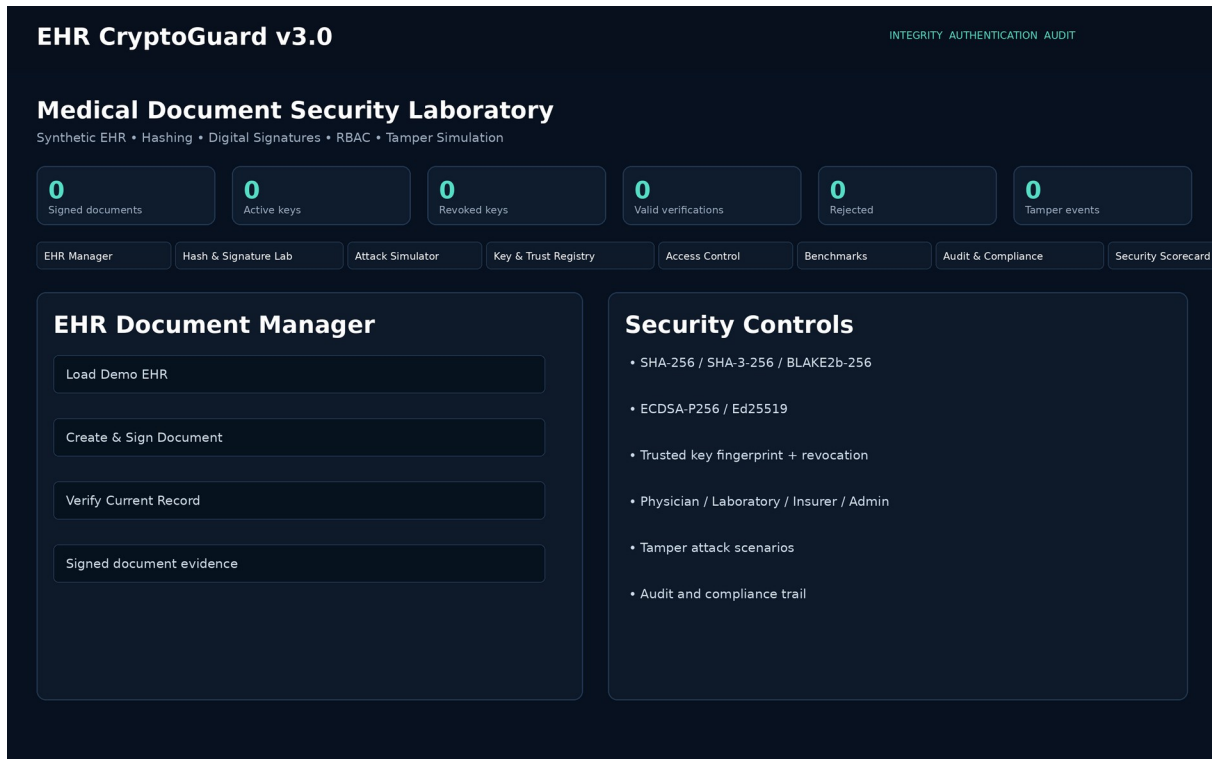


Figure 3. EHR CryptoGuard v3 browser interface.

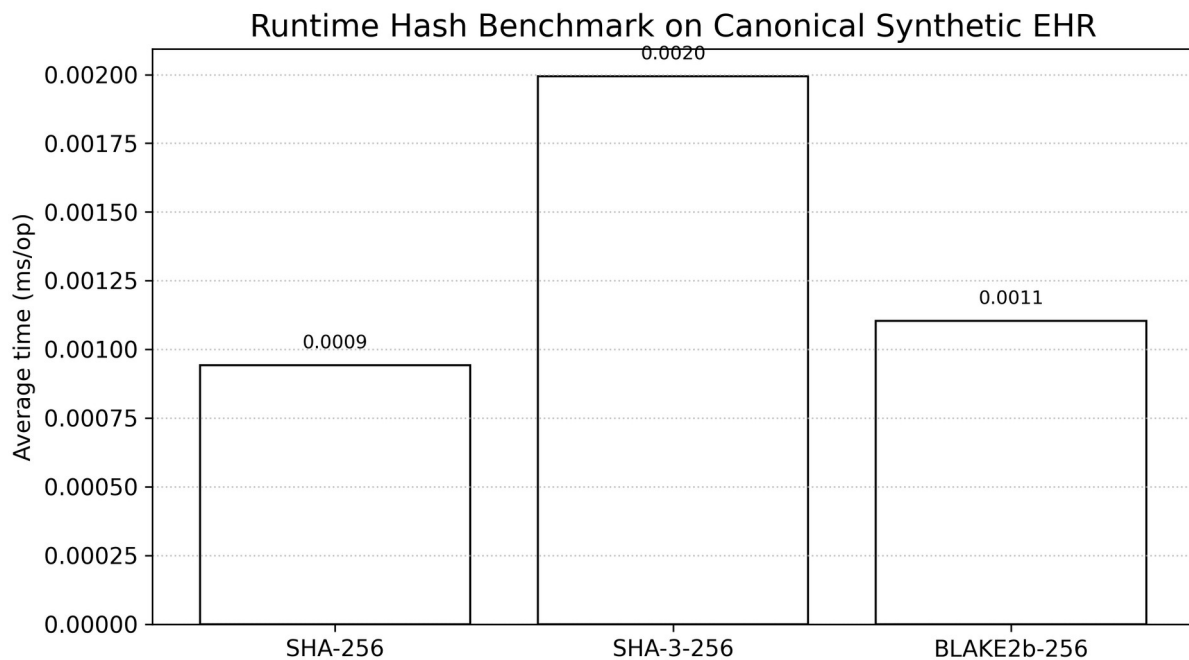


Figure 4. Hash benchmark generated by the implementation.

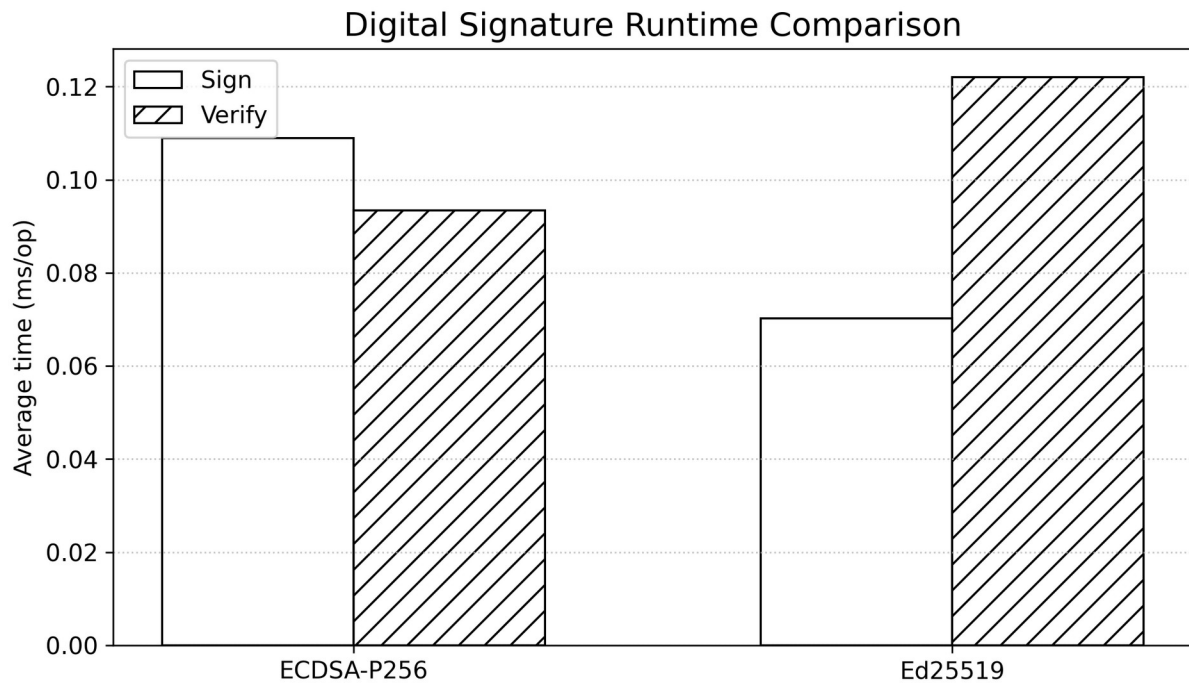


Figure 5. Digital signature benchmark generated by the implementation.

11. Results and Validation

Hash algorithm	Digest bits	Average ms/op	Kops/s
SHA-256	256	0.000943	1059.951
SHA-3-256	256	0.001994	501.56
BLAKE2b-256	256	0.001104	906.193

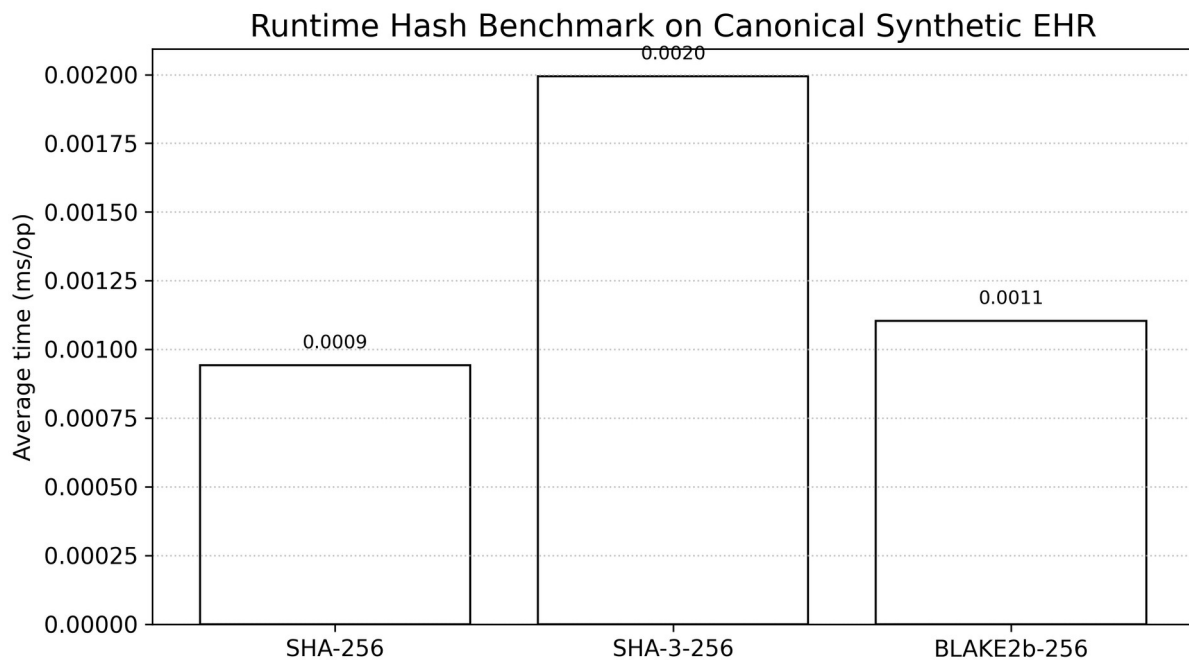


Figure 6. Runtime comparison of the three hash functions on the canonical synthetic EHR.

Signature scheme	Sign ms/op	Verify ms/op	Signature bytes	Public key bytes
------------------	------------	--------------	-----------------	------------------

ECDSA-P256	0.10897	0.09339	70	91
Ed25519	0.07017	0.12206	64	44

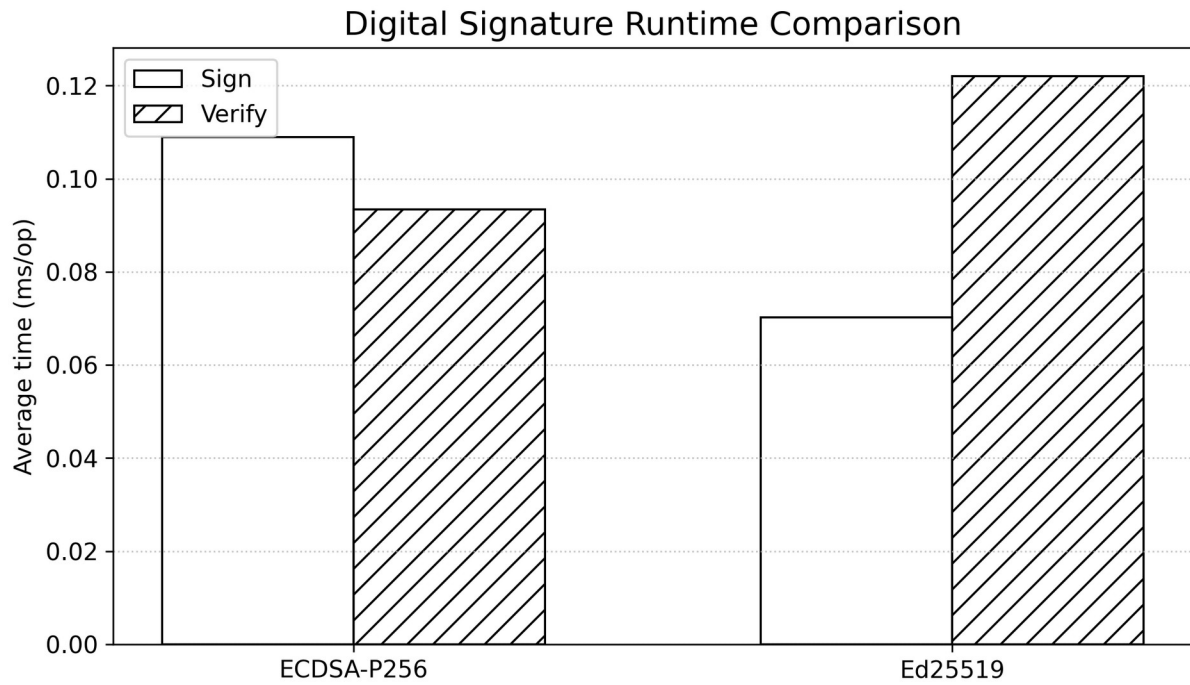


Figure 7. Runtime comparison of ECDSA-P256 and Ed25519.

Hash	Changed bits	Digest bits	Change %
SHA-256	129	256	50.39
SHA-3-256	126	256	49.22
BLAKE2b-256	120	256	46.88

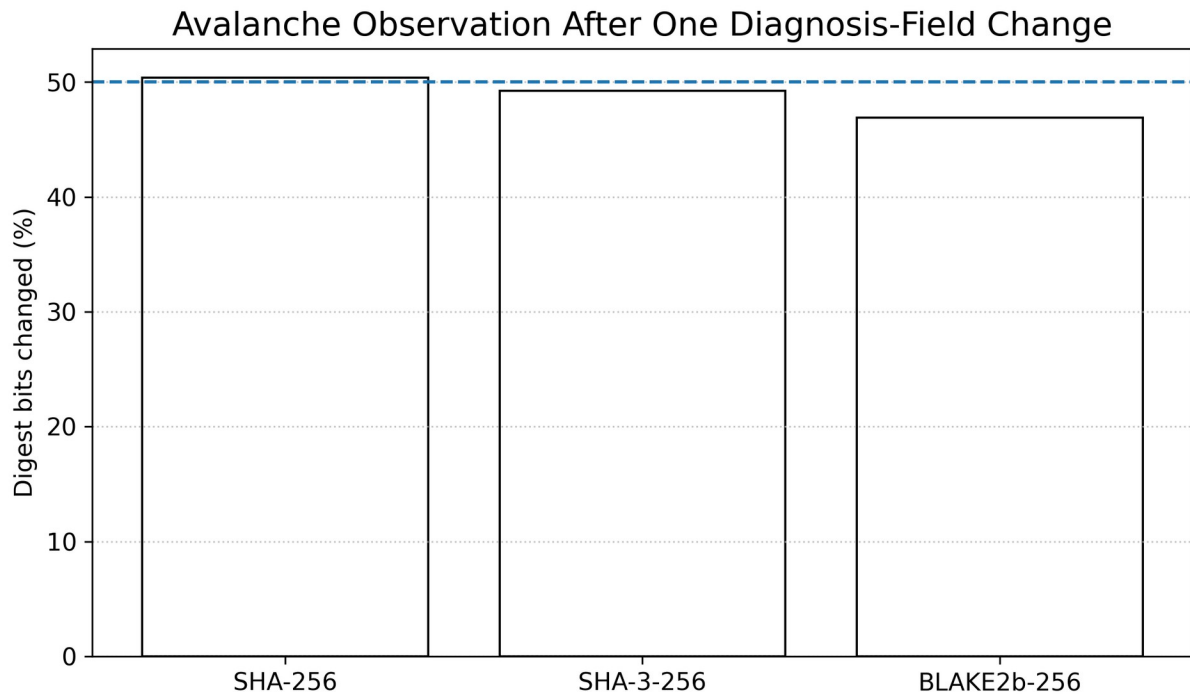


Figure 8. Avalanche observation after changing the diagnosis field.

12. Analysis, Comparison, Trade-offs and Justification

The measurements are local observations rather than universal performance claims. SHA-256 has broad ecosystem support and is a conservative default for interoperability. SHA-3-256 provides a different internal construction and is useful where algorithm diversity is desired. BLAKE2b-256 is attractive when software performance is important and the ecosystem accepts it. For signatures, Ed25519 is compact and generally simple to deploy in modern software, while ECDSA P-256 has long-standing interoperability across PKI and healthcare environments.

Decision criterion	SHA-256	SHA-3-256	BLAKE2b-256
Digest size	256 bit	256 bit	256 bit
Design family	SHA-2	Keccak/SHA-3	BLAKE2
Interoperability	Very high	High	High in software
Prototype use	Default	Diversity option	Performance option
Main trade-off	Conservative ecosystem choice	Different construction; performance varies	Ecosystem support must be considered

Criterion	ECDSA-P256	Ed25519
Interoperability	Very high in PKI environments	High in modern software
Signature size	Compact	Compact
Implementation complexity	More parameter/encoding considerations	Simpler API in many libraries
Production trust dependency	PKI/certificate lifecycle	Public-key trust lifecycle
Prototype recommendation	Best interoperability option	Best modern software option

Final protocol recommendation: use SHA-256 plus a standards-aligned signature mechanism where interoperability is the dominant requirement; use Ed25519 where the participating ecosystem supports it and compact, straightforward signatures are beneficial. The deployment should be crypto-agile so that the signature layer can be migrated to post-quantum standards as healthcare systems plan for long-lived records.

13. Research Context and Security Mapping

Recent healthcare-security literature increasingly combines cryptography with access control, decentralized audit mechanisms and post-quantum migration. Latif et al. (2026) describe an e-health authentication framework combining RBAC/ABAC, multi-factor authentication and post-quantum signatures. Lilhore et al. (2026) discuss post-quantum and zero-trust architecture for healthcare fog environments. A 2026 CITADEL study combines ML-KEM, ML-DSA, privacy controls and distributed EHR processing. These works motivate the prototype's separation of identity, authorization, integrity and future crypto-agility rather than treating hashing as a complete security solution. (Latif et al., 2026; Lilhore et al., 2026; Segar & Vijayan, 2026).

Recent 2025 work on decentralized EHRs and blockchain-supported healthcare security also emphasizes tamper resistance, privacy and auditable sharing. The prototype does not reproduce those architectures; it adopts only the relevant design lesson that integrity evidence and auditability should remain independently observable. (Ben Othman & Getahun, 2025; Ullah et al., 2025; Zhang et al., 2025).

For healthcare document signatures specifically, ISO 17090-4:2026 addresses digital signatures and certificate-enabled exchange of healthcare information. For future migration, NIST's 2026 status report documents continued evaluation of additional post-quantum digital signature schemes. The prototype therefore labels ML-

DSA/PQC as a roadmap rather than claiming that ECDSA or Ed25519 are quantum-resistant. (International Organization for Standardization, 2026; Moody et al., 2026).

14. Broader Considerations / SDG Relevance

Area	Consideration
SDG 3 – Good Health and Well-being	Integrity protection reduces the risk of altered clinical information being accepted as genuine.
SDG 9 – Industry, Innovation and Infrastructure	The prototype demonstrates interoperable cryptographic controls and algorithm benchmarking for digital health infrastructure.
SDG 16 – Peace, Justice and Strong Institutions	Audit records, signer evidence and role controls support accountability and traceability.
Privacy	Only synthetic data are used; production systems need minimization, encryption and governed access.
Ethics	Cryptographic validity does not prove that clinical content is medically correct; human clinical review remains necessary.
Safety	A failed verification should result in quarantine/rejection rather than silent acceptance.

15. Limitations and Possible Improvements

- The trust registry is in memory and is not a production certificate authority.
- Private keys are generated for demonstration and are not protected by an HSM.
- There is no authenticated user session, MFA or secure production identity provider.
- The audit log is not a tamper-evident distributed ledger.
- Performance measurements depend on the execution machine and Python/library versions.
- The prototype does not implement ML-DSA; it only documents a post-quantum migration path.
- Production deployment would require encryption at rest/in transit, secure backups, certificate policies, incident response and compliance review.

Possible improvements include HSM-backed keys, X.509/PKI integration, secure database persistence, signed audit logs, certificate revocation checking, MFA, TLS configuration, algorithm-agility interfaces and a hybrid classical/PQC signature mode for long-retention records.

16. Individual Contribution of Group Members

Use the following contribution record and replace the placeholders with the actual team allocation before submission. Do not claim work that was not performed.

Member	Contribution
Member 1	Problem formulation, cryptographic design and report analysis
Member 2	FastAPI implementation, key registry and verification workflow
Member 3	UI, attack simulator, RBAC and audit interface

17. One-Page Individual Reflection

This assignment helped me understand that protecting a digital medical record is not achieved by applying a hash function alone. During the implementation, I treated the EHR as a structured message, canonicalized it before hashing, and then used a digital signature to connect the resulting digest to a provider key. The most useful practical observation was the difference between integrity and authentication: changing a diagnosis changes the digest, but only a signature and trusted public key provide evidence about the signer. I also learned why key management is part of the security design rather than an optional add-on.

A major implementation challenge was making the verification process understandable instead of hiding it inside a library call. I therefore added a verification centre that shows the current digest, expected digest, signature result and trusted-key status. The attack simulator made the security property easier to observe because a user can alter a diagnosis, prescription or laboratory value and immediately see the rejection. The benchmark and avalanche experiment also helped me distinguish measurable implementation behaviour from theoretical security claims.

The project connects to SDG 3 because trustworthy digital records contribute to safer health information exchange, SDG 9 because it applies cryptographic engineering to digital infrastructure, and SDG 16 because auditability and accountable access are central to trustworthy institutions. If more time were available, I would integrate a real PKI, HSM-backed keys, authenticated sessions and a hybrid post-quantum signature layer. The assignment strengthened my ability to translate cryptographic theory into a testable system and to justify engineering decisions using evidence rather than assumptions.

18. Conclusion

EHR CryptoGuard demonstrates a complete academic workflow for protecting the integrity and origin of synthetic healthcare documents. The implementation combines canonicalization, three modern hash functions, two asymmetric signature schemes, trusted-key registration, revocation, RBAC, tamper simulation, benchmarks and audit evidence. The four automated cryptographic tests passed in the validation environment. The experimental results show that the choice of algorithm should consider not only local execution time but also interoperability, key management, deployment context and future migration requirements. The final design is therefore intentionally modular and crypto-agile, while clearly identifying the controls still required for production healthcare deployment.

19. References – APA 7th Edition

- Latif, R., Yakubu, B. M., Mohd Jamail, N. S., Talib, A. M., & Alomary, F. O. (2026). BBAS: A blockchain-based authentication system for e-health with multi-factor authentication, access control, and post-quantum security. *Scientific Reports*. <https://doi.org/10.1038/s41598-026-39415-5>
- Lilhore, U. K., Kumar, S., Alroobaea, R., Alsafyani, M., Baqasah, A. M., Algarni, S., & Tekeste, L. G. (2026). A unified post-quantum zero-trust architecture with AI-driven orchestration for secure healthcare fog networks. *Scientific Reports*, 16, 20144. <https://doi.org/10.1038/s41598-026-52245-9>
- Segar, N., & Vijayan, V. (2026). CITADEL: A post-quantum secure blockchain framework for privacy-preserving electronic health records with temporally-partitioned federated learning. *Frontiers in Artificial Intelligence*, 9, 1804943. <https://doi.org/10.3389/frai.2026.1804943>
- Moody, D., Alagic, G., Bros, M., Ciadoux, P., Dang, Q., Dang, T., Kelsey, J., Lichtinger, J., Liu, Y.-K., Miller, C., Peralta, R., Perlner, R., Robinson, A., Silberg, H., Smith-Tone, D., & Waller, N. (2026). Status report on the second round of the additional digital signature schemes for the NIST post-quantum cryptography standardization process (NISTIR 8610). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.IR.8610>
- International Organization for Standardization. (2026). Health informatics — Public key infrastructure — Part 4: Digital signatures for healthcare documents (ISO 17090-4:2026). <https://www.iso.org/standard/86410.html>

- Ben Othman, S., & Getahun, M. (2025). Leveraging blockchain and IoMT for secure and interoperable electronic health records. *Scientific Reports*, 15, 12358. <https://doi.org/10.1038/s41598-025-95531-8>
- Ullah, A., Ullah, Z., Rizvi, S. S., Gul, L., & Kwon, S. J. (2025). Toward blockchain based electronic health record management with fine grained attribute based encryption and decentralized storage mechanisms. *Scientific Reports*, 15, 34542. <https://doi.org/10.1038/s41598-025-17875-5>
- Tawfik, A. M., Al-Ahwal, A., Tag Eldien, A. S., & Zayed, H. H. (2025). Blockchain-based access control and privacy preservation in healthcare: A comprehensive survey. *Cluster Computing*, 28, 529. <https://doi.org/10.1007/s10586-025-05308-x>
- Zhang, et al. (2025). A decentralized and privacy-preserving framework for electronic health records using blockchain. *Alexandria Engineering Journal*, 126, 196–203. <https://doi.org/10.1016/j.aej.2025.04.069>
- Python Software Foundation. (n.d.). Python 3 documentation. <https://docs.python.org/3/>
- PyCA Cryptography contributors. (n.d.). Cryptography documentation. <https://cryptography.io/en/latest/>
- FastAPI. (n.d.). FastAPI documentation. <https://fastapi.tiangolo.com/>